Lecture 07 :
Designing a Notification System

Notification System — Swiggy / Zomato, Zepto, Instagram / fb

We start with the questions

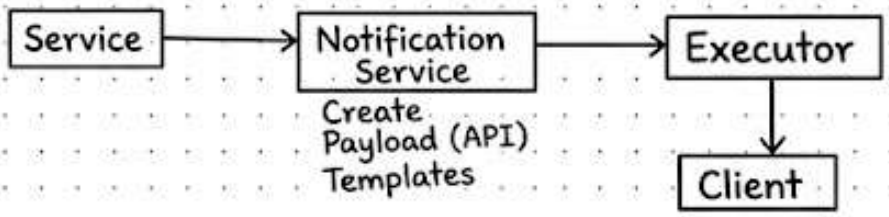
Q1. What are the types of notifications?
- ↳ push Notification (eg. someone send you a follow request or your friend sent you a reel)
- ↳ SMS Notification
- ↳ Email Notification
- ↳ In-app Notification

Real Time System vs Soft Real Time System (mostly used in real life)
- ↳ Immediately reflect the change to user (Imediately send notification to user)
- ↳ We try to send notification as soon as possible
- ↳ There can be expections
  - System is, down
  - Server is busy

Q1. Real time / soft real system?
- ↳ Soft & real time

Q3. Device type?
- ↳ IOS, Android, web (windows, macOS)

Core Entities :



Back of the Envelope Calculations

QPS , Storage Unit , Bandwidth calculations

Assumptions :

- daily Active users : 1 Million
- How many notifications are to be sent : 1 user → 10 notifications

Total notification = 1 million * 10 = 10 mill /day

Storage Unit :

1 notification = 200 bytes (content, timestamp, userId, notificationId, etc)

Total = 10 million * 200 byte

$= 10 \times 2^{20} \times 0.2 \times 10^3$

$= 10 \times 2^{20} \times 0.2 \times 2^{10}$

$= 2^{30} \times 2$

$= \sim 2\,GB$

Notification store for 1 month = 2GB x 30 days

= ~60 GB

Bandwidth Calculations :



Bandwidth : The max amount of data that can be transmitted through the network in a given time.

→ ye batati hai ki how busy our server will be

Calculation :

- Notification Payload : 1 KB
  headers, Body, request param, etc.

DAU = 1 Million

Total notification / day = 10 Million

Total data jo gaya ⇒ $1\ KB * 10\ Million$

$= 1 \times 2^{10} \times 10 \times 2^{20}$

$= 10 \times 2^{30}$

$= 10\ GB / day$

Bandwidth $= \frac{10\ million * 1\ KB}{86400}$

$= \frac{10 \times 2^{20} \times 1 \times 2^{10}}{86.4 \times 2^{10}}$

$= 0.115 \times 2^{20}$

$= 115 \times 10^{-3} \times 2^{20}$

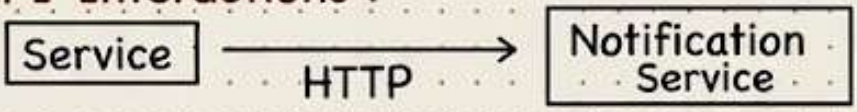
$= 115 \times 2^{-10} \times 2^{20}$

$= \sim 115\ KB/sec$

Server :
Server Scaling
Server RAM increase

i.e. if no. of Server ↑ request will distribute and each server ko kam load transfer karna padega

↳ API Interactions :



↳ Notification service kisi application ke sath judi hogi

↳ User ko jati hogi to user ka info store karta hoga (Table containing User ID, etc)

1. To retrieve user information

Get : /fetchuser/ {{userId}}

Response :

200' ok

```
{ "userId" : 101,
  "userName" : Aditya",
  "NotificationPermission" : {
      "push" : true,
      "Email" : true,
      "SMS" : false
  }
}
```

2. Send Notification

Post : {{baseURL}} / send Notification / {{userId}}

Body : or null if from system

```
{"from" : "101",
 " to" : "202",
 "Subject" : "",
 "Content" : "."
 " type" : "push/email...etc"
}
```

Response : 200 ok

```
{"Status" : "sent"
 "notificationId" "301"
}
```

3. /fetchAll Notifications → To fetch notification history (optional)

Get's

Queryparam : Filters:

date : from, to

type : email / sms etc

Response:

200 K

```
[ { "status" : "sent", "content" ; "" },
  { . . . . },
]
```

Basic Design:

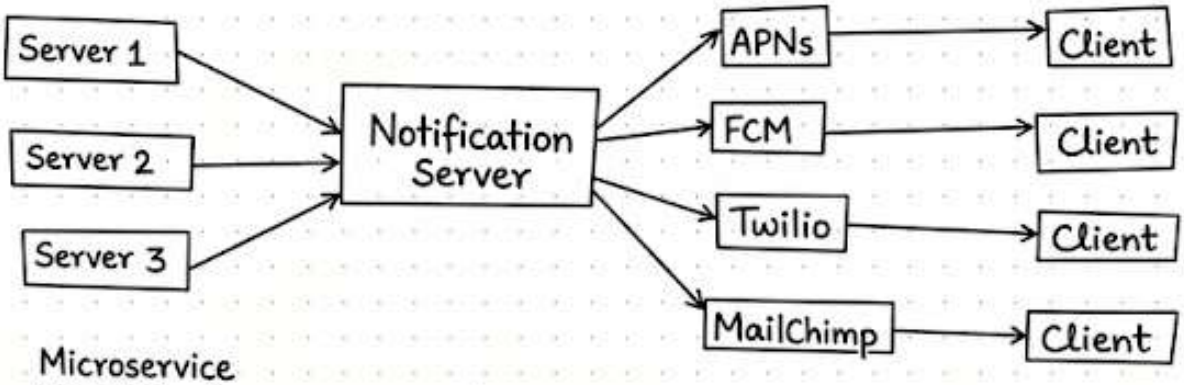
Push Notification → Mobile

Ios (Apple) → APNs (Apple push Notifications)

Android → FCM (Firebase Cloud Messaging)

SMS Notification → Twilio, nexmo, etc

Email → MailChimp, Sendgrid, etc



Microservice

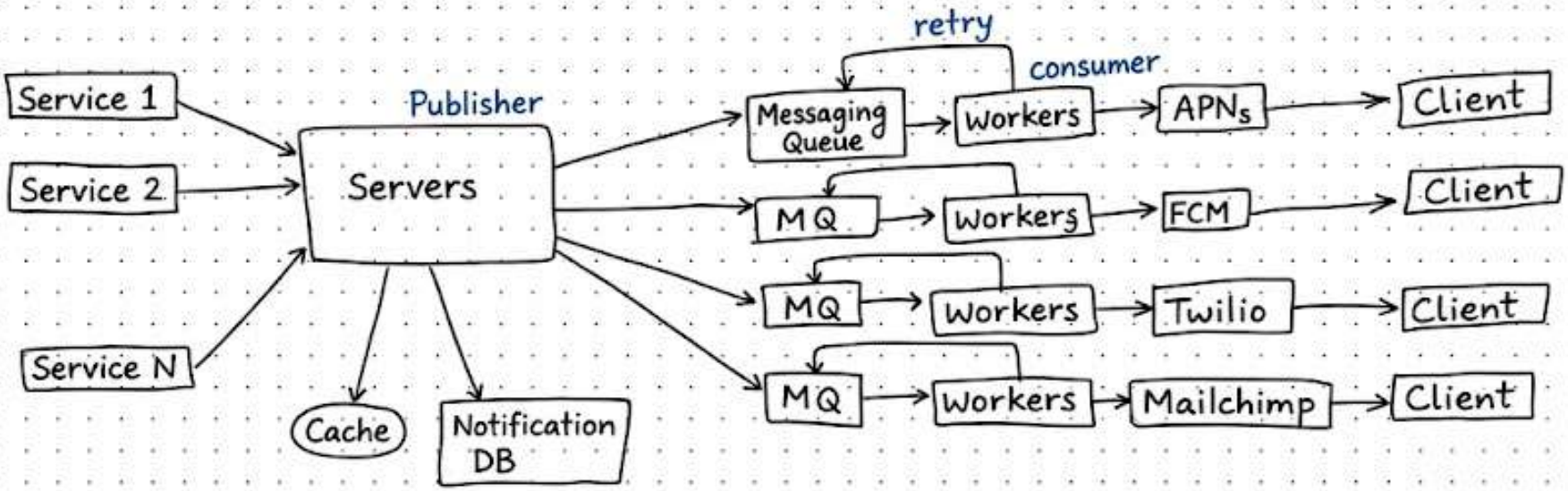
Cron-Jobs (Scheduled APIs)

Flow:

1. Any service (microservice, cronjobs, etc.) trigger Notifications and send it to our notification server
2. The notification server has APIs that builds up the Payload, for 3rd party services (APIs) to send the requested notifications
3. Third party service actually send the notification to the Client.

Problems (issues) with initial design:

1. SPOF (single point of failure):
   I need to scale the Notification server
2. Hard to scale
3. Retry mechanism missing (system should be reliable)
4. Tight Coupling (Synchronous)
   Shift to ansyn

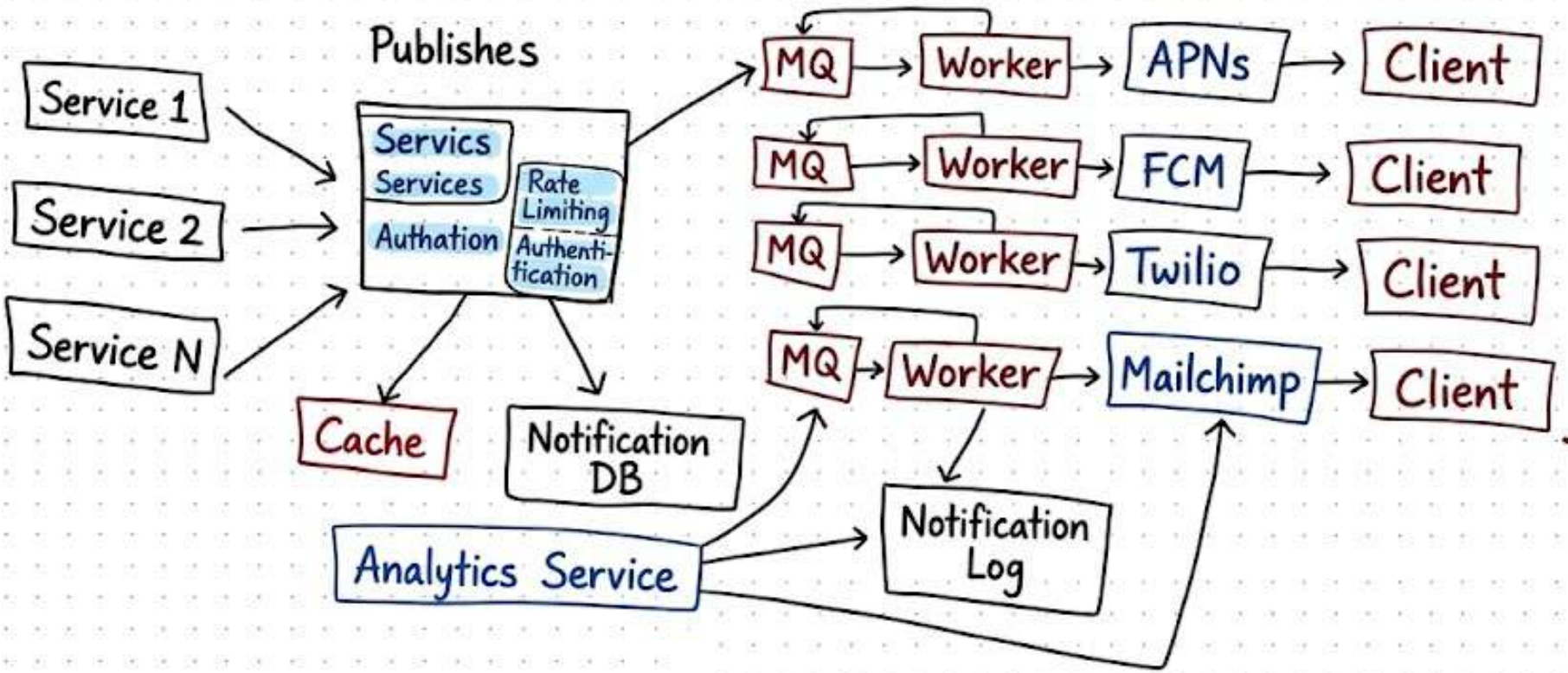


Cache & DB: User info, Notification templates etc.

MQ : MQ: Removed dependency btw 3rd party APIs and our notification async service is async service is now asyne

Improvements:

1. Notification Templates
2. Notification settings on user level:
   user can a tunon a /off a particular type of notification.
3. Rate limiting
4. API authentication
5. Monitoring Logic
6. Notification Log


Publishes
Service 1
Service 2
Service N
Servics
Services
Authation
Rate Limiting
Authenti-fication
Cache
Notification DB
Analytics Service
Notification Log
MQ
Worker
APNs
Client
MQ
Worker
FCM
Client
MQ
Worker
Twilio
Client
MQ
Worker
Mailchimp
Client
Publisher System Architecture